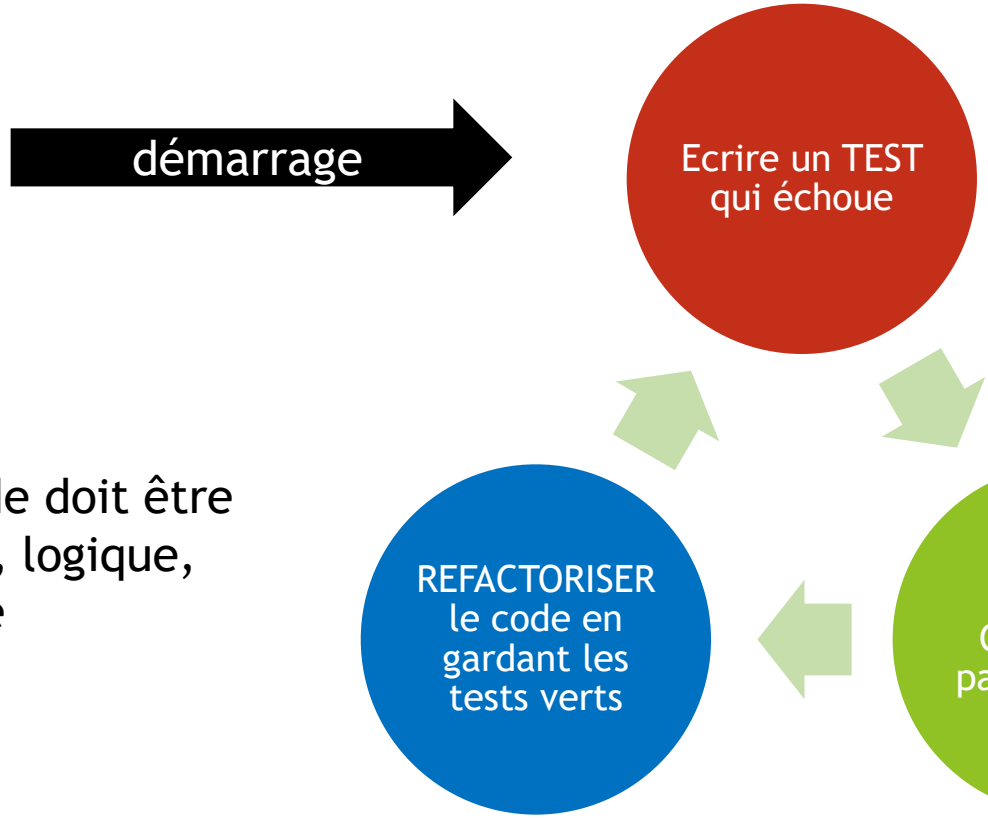
Test Driven Developement

Problématiques

- Lorsqu'une fonctionnalité est développée, la tentation est forte de deviner les besoins futurs, et de développer « au cas où »
  - + il y a de code, + il y a de bogue… comment éviter cet écueil ?
  - Présence de « code mort », gène la lisibilité, complexifie les algorithmes sans bénéfice
- Lorsque je travaille sur une fonctionnalité compliquée, je me perds et je ne sais plus où j'en suis… je suis en plein « effet tunnel »
  - J'ai du mal à évaluer ce qu'il me reste à faire et où j'en suis
  - Comment découper mes développements ?
- Au fur et à mesure du temps, l'ajout de fonctionnalité fait grossir la base de code
  - Comment s'assurer que les fonctionnalités précédentes fonctionnent toujours ?
  - Si je touche à du code historique, comment je m'assure que je n'ai rien cassé ?

Classification : Confidentiel

Principe du TDD



Traduction des besoins métiers en contrainte de code
Il faut être le plus intelligent possible

Ecrire le minimum de code.
Il faut être le plus bête possible

Le code doit être lisible, logique, propre

Classification : Confidentiel

Bénéfices

- On écrit le minimum de code
- Les tests peuvent être considérés comme de la documentation sur le comportement attendu
- Il est possible de réorganiser le code sans avoir peur de casser des fonctionnalités existantes

Classification : Confidentiel

Risques

- On écrit un test qui est vert
  - On ne peut s'assurer que c'est le code que l'on va écrire qui va faire passer le test au vert : dans ce cas il ne faut pas écrire de code !
- La qualité du code produit dépend du soin qui est donné à l'écriture des tests
- Il est possible de calculé un taux de couverture du code par les tests (% du code couvert par des tests).
  Chercher à atteindre 100% fera écrire des tests défensifs APRES l'écriture du code, ce qui rendra la refactorisation difficile
- Il ne faut surtout pas négliger l'étape de refactorisation au risque de laisser du code de mauvaise qualité

Classification : Confidentiel

Exercice : FizzBuzz !

- Ecrire un programme qui affiche tous les nombres entre 1 et 100 avec les exceptions suivantes, il affiche :
  - « Fizz » à la place du nombre si celui-ci est divisible par 3
  - « Buzz » à la place du nombre si celui-ci est divisible par 5
  - « FizzBuzz » à la place du nombre si celui-ci est divisible par 3 et par 5

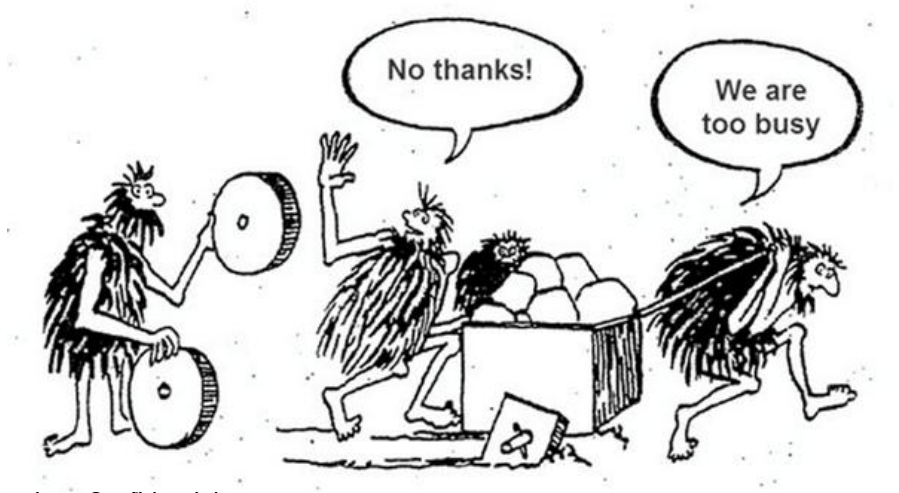
Refactoriser / Clean Code

- L'intention doit apparaître elle-même pour les autres développeurs
  - Noms de variable / méthode / classe incompréhensible ou générique, méthodes et classes trop longues, « magic number »…
- Ne pas contenir de duplication
  - Règles des 3 : on autorise 2 duplications, à la 3e on doit se poser la question
- Tous les tests doivent passer
  - Paramétrer la CI pour bloquer une PR qui ne passerait pas les tests
- Résultat : du code propre est plus facile et moins cher à maintenir

Classification : Confidentiel

Notion de dette technique

- Tout ce qui va rendre plus cher de nouveaux développements. La dette peut être intentionnelle ou non :
  - Intentionnelle : Sauter une étape de refactorisation pour aller plus vite, le code sera moins lisible et il sera plus difficile d'y revenir
  - Non intentionnelle : mise à jour de bibliothèque, framework qui devient obsolète avec le temps



Principales causes du grossissement de la dette technique

- Business
  - Pression du business : sortir des fonctionnalités
  - Manque de connaissance sur les conséquences d'une dette technique trop importante
- Technique
  - Manque de refactorisation
  - Incompétence
  - Manque de veille
- Organisationnel
  - Manque de partage entre les membres de l'équipe

Classification : Confidentiel

Tests unitaires
Les Moq

- Un test unitaire doit être capable de tester une fonctionnalité **indépendamment** des dépendances (bibliothèque, base de données, etc.)

- L'utilisation de Moq permet de définir le comportement d'une dépendance **sans** l'appeler réellement

```
var persistanceMock = new Mock<IPersistance>();
persistanceMock.Setup(p => p.LoadMovies()).Returns(movies.ToArray());
```

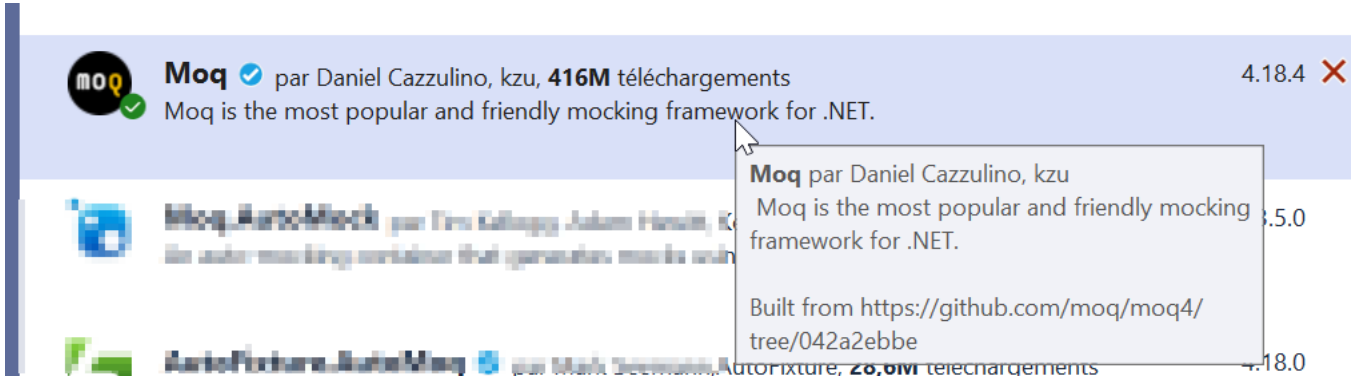
Ajout d'une définition de comportement

Comportement de la dépendance que l'on veut définir

Définition du retour

C# et Moq

- Il existe plusieurs bibliothèques pour la définition de Moq, chacune ayant sa façon de définir les éléments à vérifier et méthodes à redéfinir
  La plus répandue en C# est Moq



Définition de Moq

- Il ne faut pas définir toutes les méthodes des dépendances, seulement celles qui vont être utiles pour le test précisément

- Il est possible de définir des comportement différents selon les paramètres

```
var catalogFinderMock = new Mock<IMovieCatalogFinderService>();
//Comportement si "id1"
catalogFinderMock.Setup(c => c.Get("id1")).Returns(new Movie() { Title = "Movie 1"});
//Comportement si "id2"
catalogFinderMock.Setup(c => c.Get("id2")).Returns(new Movie() { Title = "Movie 2" });
//Comportement sinon
catalogFinderMock.Setup(c => c.Get(It.IsAny<string>())).Returns(new Movie());
```

Vérification de Moq

- Il est possible de vérifier si une méthode a bien été appelée lors du test, et si elle ne l'est pas une erreur sera remontée

```
var persistanceMock = new Mock<IPersistance>();
persistanceMock.Setup(p => p.SaveMovies(It.IsAny<Movie[]>())).Verifiable();
VideothequeService service = new VideothequeService(catalogFinderMock.Object, persistanceMock.Object);

service.AddMovieById("xyz");

persistanceMock.Verify();
```